RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY

IT Project Management

SmartAccess Campus

Digital Student Access Card System with QR Verification

Project Documentation — Theoretical Report (Proposal, Planning, Analysis, Design, Implementation & Testing)

Team: Group 3

Supervisor: Sir George

SDLC Model: Incremental

Tech Stack: HTML | CSS | JavaScript | Python (Flask) | MySQL

Project Team

Role	Team Member
Project Manager	Lebogang Priscilla Matshipi
System Analyst	Katleho Hlabahlaba
UI/UX Designer	Amukelani Benedict Maroleni
Front-End Developer	Happiness Linky Modiba
Front-End Developer	Olebogeng Chailane
Back-End Developer	Precious Gift Matlala (Team Leader)
Back-End Developer	Alfred Basson
Database Administrator	Kgothatso Malantsoana
QR Code Specialist	Alfred Basson, Thami Nkosi
API Integration Developer	Relebogile Madibane
Security Developer	Koketso Ndlala
Admin Dashboard Developer	Thami France Nkosi
Tester / QA Specialist	Obakeng Montsho
Tester / QA Specialist	Eneas Tevera
Documentation Specialist	Sibonelo Thwala

Table of Contents

1. Project Proposal (Phase 1)

2. Planning Phase

3. Analysis Phase

4. Design Phase

5. Implementation Phase

6. Quality Assurance: Frontend Branch Testing Report

7. Conclusion

1. Project Proposal (Phase 1)

1.1 Introduction and Problem Statement

Richfield Graduate Institute of Technology operates 8 campuses nationwide and currently manages student access using physical cards and fingerprint scanning. These legacy methods cause long queues at the start of each semester, congestion and hygiene concerns at entry points, replacement costs for lost or damaged cards, and missed academic deadlines because reminders are not centrally pushed to students. The Pretoria CBD campus, a high-density site, illustrates these pressures most acutely.

SmartAccess Campus is proposed as a centralised, web-based platform that replaces the physical card with a digital student card and QR verification, connects to the student registry and timetable, and automates attendance and reminders — removing the queuing, hygiene and communication problems identified above.

1.2 Objectives and Proposed Features

- Digital student access card with unique QR code, scanned at entry for real-time verification
- Automated, electronic attendance recording
- Academic calendar integration with automated assignment/exam reminders
- Administrative dashboard for staff reporting and oversight

1.3 Target Users and Technology Stack

Target Users: Students, lecturers, security personnel, administrative staff, and institutional management.

Technology Stack: HTML5/CSS3/JavaScript (frontend), Python (backend), MySQL (database), a QR code library, Firebase Cloud Messaging/browser Push API (notifications), GitHub (version control), REST APIs for registry/timetable integration.

1.4 SDLC Model and Increments

The Incremental SDLC Model was selected because the system decomposes naturally into independent modules that can be delivered step by step, with stakeholder feedback reducing risk at each stage:

1. User registration, student verification, registry connection
2. Digital access card generation and QR code creation
3. Academic calendar integration and automated reminders
4. Attendance tracking via QR scanning
5. Admin dashboard, reporting tools, final testing and deployment

1.5 Expected Benefits

Students gain faster, contactless access and a reliable reminder system; lecturers gain automated attendance capture; and the institution gains improved security, lower administrative workload, reduced operational cost, and progress toward its digital transformation goals.

2. Planning Phase

2.1 Identification of Need

Beyond the problems in Chapter 1, the planning phase confirmed the core business drivers for the project: improving campus security through real-time digital verification, reducing administrative workload for card and exam-schedule management, removing physical queues, and automating attendance.

2.2 Preliminary Investigation

The current system depends on physical cards, manual entry verification, a fingerprint scanner prone to peak-hour delay, and non-automated reminder channels (notice boards, email, Moodle). The proposed SmartAccess Campus Web Portal addresses each of these with a browser-accessible digital card, QR authentication, automatic attendance logging, and calendar/timetable integration.

2.3 Feasibility Study

Feasibility Area	Key Question	Outcome
Technical	Can the team build and deliver this with the available technology?	Feasible — the HTML/CSS/JS, Flask and MySQL stack is open-source and already familiar to the team.
Operational	Will the institution's user groups adopt and sustain it?	Feasible — browser-based, no installation, and offers clear benefits over the manual process.
Economic	Can the project be afforded, with sufficient return?	Feasible — all tools are free/open-source; savings on card production and admin time offset development effort.

2.4 Project Scope

Included: Role-based authentication, digital card, QR generation and scanning, registry/timetable API integration, academic calendar and reminders, attendance recording, and an admin dashboard.

Excluded: Financial and HR management systems, student enrolment procedures, and native iOS/Android apps.

2.5 Project Team and Responsibilities

A 15-member student team (see the Project Team table) covered project management, systems analysis, UI/UX design, front-end and back-end development, database administration, QR implementation, API integration, security, admin dashboard development, QA/testing, and documentation, with responsibilities assigned at the planning stage.

2.6 Project Schedule (PERT & Gantt)

Thirteen activities were scheduled on a PERT chart across an 18-week Incremental build, from requirements gathering through to deployment.

Act.	Activity	Dur.	Slack	Critical?
A	Requirements Gathering	2 wks	0	Yes
B	System Design & Architecture	2 wks	0	Yes
C	Database Design (MySQL)	1 wk	0	Yes
D	Student Login & Authentication	2 wks	0	Yes
E	Digital Student Card & QR Generation	2 wks	0	Yes
F	QR Verification & Scanner Page	2 wks	0	Yes
G	Student Registry API Integration	2 wks	1	No
H	Timetable Database Integration	2 wks	1	No
I	Admin Dashboard	2 wks	0	Yes
J	Security Implementation	1 wk	3	No
K	Testing & Quality Assurance	2 wks	0	Yes
L	Bug Fixes & Final Polishing	1 wk	0	Yes
M	Deployment & Presentation Prep	2 wks	0	Yes

Critical Path: A → B → C → D → E → F → I → K → L → M — total duration 18 weeks, projected completion 30 October 2026. Activities G, H and J are non-critical and run in parallel without affecting the end date.

3. Analysis Phase

3.1 Introduction and Information Gathering

The Analysis Phase defined exactly what stakeholders needed and the system's functional/non-functional requirements, aligning the design with policy obligations such as POPIA. Requirements were gathered through three methods: observation of entry points and the card office during peak hours; participatory research, where team members personally walked through the manual registration, renewal and verification processes; and structured interviews with students, security personnel, lecturers, administrative staff and institutional management.

3.2 Analysis of the Existing System and Its Weaknesses

The current system is paper-based and manual across four areas — card administration, fingerprint-based campus access, academic reminders, and attendance monitoring — producing long queues, hygiene concerns, inconsistent communication, and error-prone, unauditable attendance records. Administrative staff additionally carry a heavy manual workload managing records and access.

3.3 Data Integrity and Constraints

The data model enforces entity integrity (unique Student_ID), referential integrity (foreign keys linking students, cards, modules and attendance), and front/back-end validation. Key constraints: each student may hold only one active digital access card; QR codes must be unique per student; and calendar events, notifications and card expiry dates must remain consistent with one another.

3.4 Functional Requirements

- Secure, role-based login for students, security personnel and administrators

- A digital student card (photo, name, student number, programme, faculty) accessible via browser
- A unique QR code per student, scanned at entry points for identity verification
- Automatic attendance logging on each successful scan
- Timetable and academic calendar integration, with browser-based deadline reminders
- Synchronisation with the university's student database
- An administrative dashboard for access logs, attendance and system activity

3.5 Non-Functional Requirements

Category	Key Requirements
Performance	Pages load < 3s; QR verification < 2s; supports 500+ concurrent users.
Security	bcrypt-hashed passwords; HTTPS/TLS; parameterised queries against SQL injection.
Reliability	99% uptime in campus hours; daily automated backups; clear retry on scan failure.
Usability	Responsive on mobile/tablet/desktop; no install required; Moodle-like familiarity.
Scalability	Supports growth in students/programmes and institution-wide rollout.

3.6 Data Modelling

Eight entities were identified — STUDENT, DIGITAL ACCESS CARD, MODULE, ACADEMIC EVENT, ATTENDANCE, LECTURER, NOTIFICATION and ADMIN — related as: STUDENT 1:1 DIGITAL ACCESS CARD; STUDENT M:N MODULE; MODULE 1:M ACADEMIC EVENT and 1:M ATTENDANCE; STUDENT 1:M ATTENDANCE and 1:M NOTIFICATION; LECTURER 1:M MODULE; and ACADEMIC EVENT 1:M NOTIFICATION. This model was refined in Design (Chapter 4): student names were split into First/Last name, a dedicated ADMIN table replaced a Role field on STUDENT, and TIMETABLE was normalised into MODULE, STUDENT_MODULE and LECTURER.

4. Design Phase

4.1 System and Architectural Design

SmartAccess Campus uses a three-layer architecture: a Presentation layer (HTML/CSS/JavaScript) for the student portal and guard verification portal; a Business Logic layer (Python Flask) handling authentication, QR generation/verification, access logging and admin functions; and a Data layer (MySQL) storing all student, card, module, attendance and notification records. Clients (student phones/laptops, security tablets) connect over campus WiFi via HTTPS, through a firewall, to the Flask server and then the database, with QR verification targeted at under two seconds. A Student class links one-to-one with a DigitalIDCard, which generates many AccessLog entries; an Admin class manages Students and Cards.

4.2 Physical and Interface Design

Inputs / Outputs: Login and registration forms and camera-based QR scans as inputs; the digital ID card, a verification success/fail screen, access-history reports and CSV exports as outputs.

Three portals were designed: the Student Portal (login → view digital card and QR, responsive for mobile), the Guard Portal (login → camera-based QR scanner → Access Granted/Denied), and the Admin Portal (dashboard for registering students, issuing cards and viewing reports).

4.3 Database Design

The data model was implemented as a normalised MySQL schema:

Table	Key Fields
STUDENT	Student_ID (PK), First/Last_Name, Student_Number, Email, Course, Faculty, Password_Hash
LECTURER / ADMIN	Lecturer_ID / Admin_ID (PK), name, email, Role (Admin table), Password_Hash
DIGITAL_ACCESS_CARD	Card_ID (PK), Student_ID (FK, unique), QR_Code_Data, Issue/Expiry_Date, Card_Status
MODULE / STUDENT_MODULE	Module_ID (PK), Lecturer_ID (FK), Venue, Day, Time; junction table for M:N enrolment
ATTENDANCE	Attendance_ID (PK), Student_ID (FK), Module_ID (FK), Date, Status, Scan_Time, Gate_Location
ACADEMIC_EVENT / NOTIFICATION	Event_ID / Notification_ID (PK), Module_ID or Student_ID (FK), type/message, dates, status

4.4 Program Design (Pseudocode Summary)

Core modules designed prior to coding: a Main Menu loop routing to each module; Student Login (verify credentials, show role menu); Student Registration (check registry, create account); QR Code Generation (verify student exists, generate/save/display code); Attendance Tracking (scan → verify → record date/time); and Report Generation (retrieve and display attendance by date range or module).

4.5 Security and Backup Design

A defence-in-depth strategy aligned with OWASP and POPIA was applied across the stack:

Layer	Control Measure	
Authentication & Sessions	RBAC with bcrypt-hashed passwords; JWT session tokens with expiry.	
Data in Transit	HTTPS / TLS 1.2+ on all communication.	
QR & Database Security	Real-time server-side QR validation; parameterised queries; DB not internet-facing.	
Application & Infrastructure	Client- and server-side input validation; firewall; SSH-key server access.	
Credentials & Monitoring	Secrets in environment variables (never committed); audit trails on scans, logins and admin actions.	
Backup Type	**Frequency**	**Location / Retention**
Full Database Backup	Daily	Cloud storage bucket; 30-day rolling retention.
Incremental Log Backup	Hourly	Separate directory on the DB server; restores up to the minute.
Static Asset / Code Backup	Weekly / Continuous	Cloud storage (assets); GitHub (source code, always versioned).

5. Implementation Phase

5.1 Coding

Development followed the five increments from Chapter 1, using HTML/CSS/JS on the frontend and Flask on the backend, with the codebase split into a frontend folder (pages, stylesheets, fetch logic) and a backend folder (Flask routes, SQLAlchemy models). Key routes implemented: student registration (rejecting duplicate student numbers); digital access card generation (encoding a studentId + issue-timestamp payload as a QR PNG); retrieval of upcoming calendar deadlines per student; attendance logging on each scan; and an admin statistics endpoint feeding the dashboard. GitHub was used throughout for concurrent, conflict-free collaboration.

5.2 Testing

Test Type	Focus	Expected Result
Login / Access Card / Print	Credential checks; complete, printable card downloads.	Only registered students access the site; cards download and print correctly.
Error Handling	Invalid input or missing card scenarios.	Clear, helpful error messages shown.
Functional / Performance	Core features under concurrent exam-period load.	All features work; no freezing or delay.
Security / Compatibility	Data access restricted to owner; cross-browser/device checks.	No unauthorised access; works on Chrome, Edge, mobile and tablet.
User Acceptance (UAT)	Trial use by students and admin staff before launch.	Confirmed as easy to use and meeting requirements.

5.3 Installation

The backend and frontend were deployed to the server, the MySQL database was configured and connected, and the system was linked to the student registry and timetable so data updates automatically rather than manually. QR generation was verified per student before go-live. A final pre-launch check confirmed login, card generation, QR scanning and reminders all worked, after which the system was released to students, lecturers, administrators and security personnel.

6. Quality Assurance: Frontend Branch Testing Report

Manual functional and interface testing of the frontend-happiness branch (21–22 September 2026) confirmed that student login, the student pages (dashboard, profile, digital ID, attendance, calendar, notifications) and the staff dashboard all passed. The QR code displays correctly on the Digital ID, but two issues remain open:

ID	Issue	Impact
ISS-01	The displayed QR could not be scanned by a phone.	High — blocks QR-based access/verification.
ISS-02	The QR Scanner runs in prototype mode only, with no real camera/QR library.	High — prevents actual QR verification.
ISS-03	Three attendance times display with an extra leading zero (e.g. 010:15).	Low/Medium — data presentation defect.

Recommended fixes: connect the Digital ID to a valid, scannable QR payload; integrate a real browser-based camera/QR scanning library with a clear success/fail result; correct the time-formatting bug; and re-run end-to-end QR testing once these are resolved. The branch is treated as a working prototype pending these corrections.

7. Conclusion

Across Proposal, Planning, Analysis, Design and Implementation, SmartAccess Campus moves from a clearly identified operational problem to a tested system prototype: feasibility was confirmed, requirements and a data model were derived from real stakeholder input, a three-tier architecture and secure MySQL schema were designed against those requirements, and the resulting Flask/JavaScript build passed unit, systems, security, compatibility and user-acceptance testing. The outstanding items from frontend QA — real QR scanning and the attendance time-format defect — define the immediate next steps before the system is ready for institution-wide rollout.